

DAT22 - 5

SQL & Data Querying

[Module page](#)

Version of 15 September 2026

About this textbook

This year, for the first time, every Bachelor course at Albert School has its own textbook, thanks to your teachers and AI.

- The textbook is a help for working on the course at home, after the class.
- The course as taught in class remains the reference, and prevails over the textbook wherever the two differ.
- This is a first version. Help us improve it by sharing your feedback with your teacher.

Contents

Preface	4
1. The relational model and your first SELECT	5
1.1. Why tables, and why more than one	5
1.2. The course database	6
1.3. The anatomy of a SELECT	8
1.4. WHERE: filtering rows	9
1.5. ORDER BY and LIMIT: sorting and trimming	9
1.6. GROUP BY and HAVING: from rows to groups	10
2. Aggregate functions	12
2.1. One number from many rows	12
2.2. Choosing the right aggregate	13
3. Joins	15
3.1. Putting the split-up tables back together	15
3.2. The four join types	15
3.3. Which join, and why it matters	17
3.4. Joining more than two tables	18
4. Subqueries and CTEs	20
4.1. When one query is not enough	20
4.2. Subqueries	20
4.3. Common Table Expressions (CTEs)	22
5. Creating and manipulating tables	24
5.1. Defining where the data lives	24
5.2. Inserting, updating, deleting	25
6. Database types: relational, warehouse, flat file	27
6.1. Different jobs, different stores	27
7. Connecting to SQLite from Python	29
7.1. SQL where you already work	29
8. Data quality at the query level	31
8.1. Clean the data where it lives	31
8.2. Duplicates	31
8.3. NULL handling	32
8.4. Inconsistent string formats	34
9. From business question to SQL	35
9.1. The translation skill	35
10. Indexing and query performance	37
10.1. When the table gets big	37
10.2. The trade-off: indexes are not free	38

Preface

By the end of this book you will take a business question asked in plain language, translate it into **SQL**, run it against a real relational database — from the command line or from Python — and read the answer back in business terms: joining and aggregating across many related tables, cleaning data as you query it, and keeping queries fast as the data grows.

Today, working in pandas, you can interrogate a single table held in memory. What you cannot yet do is query a **persistent, multi-table database** in the language nearly all of the world's structured data lives in. That is the gap this book closes — the foundation the machine-learning work ahead of you quietly assumes.

1. The relational model and your first SELECT

In your earlier work with pandas you learned to take a table — a DataFrame — and interrogate it: filter rows with a boolean mask, summarise with groupby, stitch tables together with merge, and write a paragraph saying what the numbers mean. You already think in terms of **tables, conditions, groups, and joins**. That mental model is exactly the one SQL rests on; what changes is the **medium**.

In pandas the table lived in your computer's memory, inside a Python process, and you manipulated it with method calls. Here the table lives in a **database** — a program built for one job: storing related tables and answering questions about them, even when there are millions of rows and the data outlives any single script. You ask those questions in **SQL** (Structured Query Language), the language every database understands and the shared language of data work everywhere. A data scientist who cannot read and write SQL is cut off from where most of the world's structured data actually lives.

Almost every pandas idea has a SQL twin, and naming the correspondence up front makes the work feel like translation rather than starting over:

In pandas you wrote...	In SQL you write...
<code>df[df["amount"] > 100]</code>	<code>WHERE amount > 100</code>
<code>df.sort_values("amount")</code>	<code>ORDER BY amount</code>
<code>df.groupby("customer_id").sum()</code>	<code>GROUP BY customer_id + SUM(...)</code>
<code>a.merge(b, on="id")</code>	<code>a JOIN b ON a.id = b.id</code>
<code>df["amount"].mean()</code>	<code>AVG(amount)</code>

A database also gives you what an in-memory DataFrame cannot: **persistence** (the data stays put), **multiple related tables** the database itself keeps consistent, **transactional changes** (INSERT, UPDATE, DELETE), and **indexes** that keep queries fast as the data grows — each a thread this book picks up in turn.

1.1. Why tables, and why more than one

A **relational database** stores data in **tables** (also called **relations**): a grid of rows and columns, where each row is one record and each column one named, typed attribute. So far this is just a DataFrame. The leap is that a database holds **many** tables that **refer to each other**, and does not let those references break.

Example — One table is not enough. Imagine recording online orders. You could put everything in one big table, repeating the customer's name and country on every single order:

order_id	customer_name	country	amount
1	Amara	Brazil	120
2	Amara	Brazil	300
3	Bruno	France	200

Amara’s details are copied onto every order she places. If she moves to Spain you must find and fix **every** row, and if you miss one the data now disagrees with itself. The relational answer is to split this into **two** tables — one for customers, one for orders — and let each order **point at** a customer.

Definition 1 (Primary key and foreign key). A **primary key** is a column (or set of columns) whose value **uniquely identifies** each row of a table — no two rows share it, and it is never NULL. A **foreign key** is a column in one table that **holds the primary key value of a row in another table**, creating a link between them. Foreign keys are how a relational database represents relationships without copying data.

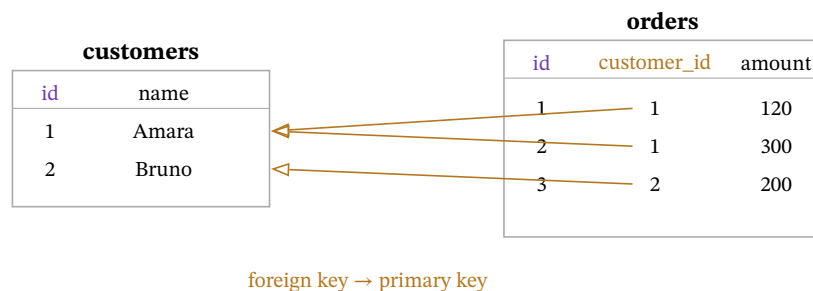


Figure 1: Two tables linked by a foreign key. `orders.customer_id` (orange) stores the value of `customers.id` (purple). Amara’s name lives in exactly one place; her two orders both point at it. Change her name once and every order sees the update.

Remark. This is the database version of **not repeating yourself**, and it is the single idea that separates “data in a database” from “data in a spreadsheet”. Splitting data into linked tables to remove redundancy is called **normalisation**; you do not need its formal theory in this course, only the instinct: each fact should live in one place, and relationships are expressed by keys, not by copying.

1.2. The course database

Every example in this book runs against one small shop database, built exactly along those lines. It has three tables: `customers` and `products` hold **facts** (who the buyers are, what the catalogue is), and `orders` records **events** — each order names its customer and product by **id**, never by copying their details. You build it yourself in a moment and query it for the rest of the book; a few later chapters spin up a tiny extra table or a second throwaway database of their own, each with its build script printed. Either way you can run every query as you read and check your result against the one printed beside it.

id	name	country	signup_date
1	Amara	Brazil	2023-02-11
2	Bruno	France	2023-06-30
3	Chen	Japan	2024-01-15
4	Diego	Spain	2024-03-22
5	Farah	France	2024-07-08

6	Gita	Brazil	2025-01-05
---	------	--------	------------

id	customer_id	product_id	order_date	amount
1	1	3	2024-05-02	120
2	1	1	2025-03-14	300
3	1	2	2025-08-22	250
4	2	4	2024-11-20	200
5	2	2	2025-02-09	450
6	2	1	2025-07-04	380
7	4	3	2024-09-12	150
8	4	5	2025-06-01	90
9	5	2	2025-04-18	600
10	5	4	2025-09-01	520
11	6	2	2025-05-30	700
12	6	1	2025-10-11	260

Notice how orders stores only a `customer_id` and a `product_id`: an order knows **which** customer and product by their keys, and nothing else about them. `customers.id` is the primary key of the customers table; `orders.customer_id` is a foreign key pointing back at it.

Remark (Build it yourself). Create the database once and keep the file; you will query it throughout the book. Save the script below as `setup.sql`, then run these three lines of Python (the `sqlite3` module is built in — no install):

```
import sqlite3
conn = sqlite3.connect("shop.db")
conn.executescript(open("setup.sql").read())
conn.close()
```

```
CREATE TABLE customers (
  id          INTEGER PRIMARY KEY,
  name        TEXT NOT NULL,
  country     TEXT,
  signup_date TEXT
);

CREATE TABLE products (
  id          INTEGER PRIMARY KEY,
  name        TEXT NOT NULL,
  category    TEXT
);

CREATE TABLE orders (
  id          INTEGER PRIMARY KEY,
  customer_id INTEGER NOT NULL REFERENCES customers(id),
  product_id  INTEGER NOT NULL REFERENCES products(id),
```

```

    order_date TEXT NOT NULL,
    amount     INTEGER NOT NULL
);

INSERT INTO customers (id, name, country, signup_date) VALUES
(1, 'Amara', 'Brazil', '2023-02-11'),
(2, 'Bruno', 'France', '2023-06-30'),
(3, 'Chen', 'Japan', '2024-01-15'),
(4, 'Diego', 'Spain', '2024-03-22'),
(5, 'Farah', 'France', '2024-07-08'),
(6, 'Gita', 'Brazil', '2025-01-05');

INSERT INTO products (id, name, category) VALUES
(1, 'Notebook', 'Stationery'),
(2, 'Backpack', 'Bags'),
(3, 'Bottle', 'Accessories'),
(4, 'Desk lamp', 'Home'),
(5, 'Pen set', 'Stationery');

INSERT INTO orders (id, customer_id, product_id, order_date, amount) VALUES
( 1, 1, 3, '2024-05-02', 120),
( 2, 1, 1, '2025-03-14', 300),
( 3, 1, 2, '2025-08-22', 250),
( 4, 2, 4, '2024-11-20', 200),
( 5, 2, 2, '2025-02-09', 450),
( 6, 2, 1, '2025-07-04', 380),
( 7, 4, 3, '2024-09-12', 150),
( 8, 4, 5, '2025-06-01', 90),
( 9, 5, 2, '2025-04-18', 600),
(10, 5, 4, '2025-09-01', 520),
(11, 6, 2, '2025-05-30', 700),
(12, 6, 1, '2025-10-11', 260);

```

1.3. The anatomy of a SELECT

A SELECT query **reads** rows from one or more tables. Nearly everything you do in this course is a SELECT. Its clauses always appear in this written order:

SELECT	columns	-- which columns to return
FROM	table	-- which table to read
WHERE	condition	-- keep only rows matching this
GROUP BY	columns	-- collapse rows into groups
HAVING	group_condition	-- keep only groups matching this
ORDER BY	columns	-- sort the result
LIMIT	n;	-- return at most n rows

A crucial subtlety, and the source of many beginner errors: SQL is **written** in the order above, but the database **evaluates** the clauses in a **different** order. It starts from the table, filters rows, forms groups, filters groups, and only **then** works out which columns to show and how to sort them.

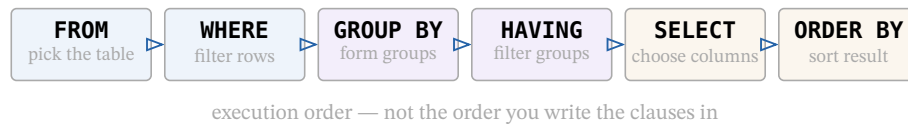


Figure 2: The logical order in which a database evaluates a query. **WHERE** filters individual rows **before** groups exist; **HAVING** filters groups **after** they are formed; **SELECT** runs late, which is why a column alias defined in **SELECT** cannot usually be reused back in **WHERE**.

1.4. WHERE: filtering rows

WHERE keeps only the rows for which its condition is true. It runs **before** any grouping, so it sees individual rows. Ask the orders table for every order worth more than 300:

```
SELECT id, customer_id, amount
FROM orders
WHERE amount > 300;
```

id	customer_id	amount
5	2	450
6	2	380
9	5	600
10	5	520
11	6	700

Conditions combine with **AND**, **OR**, and **NOT**, and use the comparison operators you already know plus a few SQL-specific ones:

Operator	Meaning
= < > <= >=	equal, not-equal, and the orderings
BETWEEN a AND b	within a range, inclusive
IN (v1, v2, ...)	equals any value in the list
LIKE 'A%'	pattern match (% = any run of characters, _ = one)
IS NULL / IS NOT NULL	test for missing values

Pitfall. SQL text strings are wrapped in **single** quotes: 'France'. Double quotes mean something different (an identifier, like a column name) in standard SQL. And equality uses a **single** =, not the == of Python. `WHERE country == 'France'` is a Python habit that many databases reject.

1.5. ORDER BY and LIMIT: sorting and trimming

ORDER BY sorts the rows the query returns. **ASC** (ascending) is the default; **DESC** reverses it. You can sort by several columns — ties on the first are broken by the second. Pair **ORDER BY** with **LIMIT** to answer “top N” questions directly: sort, then take the first few rows. The three largest orders:

```
SELECT id, customer_id, amount
FROM orders
ORDER BY amount DESC
LIMIT 3;
```

id	customer_id	amount
11	6	700
9	5	600
10	5	520

1.6. GROUP BY and HAVING: from rows to groups

GROUP BY is the heart of analytical SQL, and it is exactly the `df.groupby` you already know. It **collapses** all rows sharing the same value in the listed column(s) into a single group, so that an **aggregate function** (Chapter 2) can compute one summary number per group. Total revenue per customer:

```
SELECT customer_id, SUM(amount) AS revenue
FROM orders
GROUP BY customer_id
ORDER BY revenue DESC;
```

customer_id	revenue
5	1120
2	1030
6	960
1	670
4	240

Every order with `customer_id = 5` is folded into one group; `SUM(amount)` then totals their amounts. The result has **one row per group** — one per `customer_id` that appears in orders — not one per order. (The groups are still labelled by the opaque `customer_id`; in Chapter 3 a join will replace those ids with names.)

HAVING then filters those **groups**, using a condition on the aggregate. This is the clause beginners most often miss, because it looks like it duplicates WHERE. The distinction is sharp and worth memorising:

Definition 2 (WHERE vs HAVING). **WHERE filters rows, before grouping.** It cannot mention an aggregate, because groups do not exist yet. **HAVING filters groups, after grouping.** It is the only place an aggregate condition such as `SUM(amount) > 800` may go.

Example — The two filters working together. “Among orders placed in 2025, which customers spent more than 800 in total?” Dates stored in ISO form (2025-03-14) compare correctly as text, so `order_date >= '2025-01-01'` keeps exactly the 2025 rows.

```
SELECT customer_id, SUM(amount) AS revenue
FROM orders
WHERE order_date >= '2025-01-01'  -- row filter: keep only 2025 orders
GROUP BY customer_id
HAVING SUM(amount) > 800          -- group filter: keep big spenders
ORDER BY revenue DESC;
```

customer_id	revenue
5	1120
6	960
2	830

WHERE throws away non-2025 rows **before** totals are computed; HAVING inspects each customer’s total **after**. Swapping them is impossible: a WHERE on `SUM(amount)` is an error, and a HAVING on `order_date` would filter far too late.

Pitfall. Every non-aggregated column in the SELECT list of a grouped query must appear in GROUP BY. Writing `SELECT customer_id, product_id, SUM(amount) ... GROUP BY customer_id` is ambiguous — each customer group contains many different `product_ids`, so which one should the database show? Standard SQL rejects this; SQLite returns an arbitrary one, which is worse, because it looks like it worked. Group by every plain column you select, or aggregate it.

Exercises

1.1. On the orders table, write one SELECT that uses WHERE, GROUP

BY, HAVING, and ORDER BY together. State, for each clause, whether it acts on rows or on groups, and why it must run where it does.

1.2. Write a query returning the three **smallest** orders. Then change one clause so it returns the three **largest** instead.

1.3. Explain why `SELECT customer_id, amount FROM orders GROUP BY`

`customer_id` is a badly-formed query, and give two different well-formed queries that fix it.

Answers. (1) e.g. `SELECT customer_id, SUM(amount) AS revenue FROM orders WHERE amount >`

`100 GROUP BY customer_id HAVING SUM(amount) > 500 ORDER BY revenue DESC`; WHERE acts on individual rows before groups exist; GROUP BY forms the groups; HAVING acts on the finished groups (it is the only place `SUM(amount)` may be tested); ORDER BY sorts the surviving group rows last. (2) ... `ORDER BY amount ASC LIMIT 3` for the smallest; switch ASC to DESC for the largest. (3) `amount` is neither grouped nor aggregated, so each `customer_id` group holds many amounts and the database cannot pick one. Fix by aggregating — `SELECT customer_id, SUM(amount) ... GROUP BY customer_id` — or by grouping on it too — `GROUP BY customer_id, amount`.

2. Aggregate functions

2.1. One number from many rows

An **aggregate function** takes a whole column of values and returns a single summary value. On its own it summarises the entire table; with `GROUP BY` it summarises each group. Five of them do most of the work, and the quickest way to meet all five is to see them side by side, profiling the orders table at a glance:

Example — Profiling a column at a glance.

```
SELECT
  COUNT(*)      AS n_orders,
  SUM(amount)   AS total,
  AVG(amount)   AS mean,
  MIN(amount)   AS smallest,
  MAX(amount)   AS largest
FROM orders;
```

n_orders	total	mean	smallest	largest
12	4020	335.0	90	700

One row comes back, summarising the whole orders table: how many orders, their total, their typical size, and the extremes. This is the SQL equivalent of `pandas' df["amount"].describe()`.

Each name says what it does; the skill to build is not just using them but knowing **which one a question calls for**.

Definition 3 (The five core aggregates).

- **COUNT** — **how many** rows. `COUNT(*)` counts all rows; `COUNT(col)` counts rows where `col` is not `NULL`; `COUNT(DISTINCT col)` counts distinct non-`NULL` values.
- **SUM** — the **total** of a numeric column.
- **AVG** — the **mean** (average) of a numeric column.
- **MIN** — the **smallest** value.
- **MAX** — the **largest** value.

`SUM` and `AVG` require numbers; `COUNT`, `MIN`, and `MAX` work on any type (`MIN` of a text column is the alphabetically first value, `MIN` of a date the earliest).

`COUNT(DISTINCT ...)` answers “how many **different** values” — for instance, how many distinct countries the customers span:

```
SELECT COUNT(*) AS n_customers,
       COUNT(DISTINCT country) AS n_countries
FROM customers;
```

n_customers	n_countries
6	4

2.2. Choosing the right aggregate

Each aggregate answers a different shape of business question. Matching the question to the function is the real skill:

The question asks for...	Use
how many customers / orders / distinct countries	COUNT (with DISTINCT for “distinct”)
total revenue, total units sold	SUM
typical order value, average rating	AVG
cheapest product, earliest signup	MIN
biggest order, most recent login	MAX

Pitfall. Three classic aggregate traps:

- **COUNT(*) vs COUNT(col).** COUNT(*) counts rows; COUNT(col) ignores rows where col is NULL. To count **customers who have recorded a country**, use COUNT(country), not COUNT(*) — you will see the two disagree on a messy table in Chapter 8.
- **AVG silently skips NULLs.** AVG(amount) averages only the non-NULL amounts — it does **not** treat a missing value as zero. If a NULL should count as 0, write AVG(COALESCE(amount, 0)). The two can differ a lot.
- **Don’t average an average.** The average of per-day averages is **not** the overall average unless every day has the same number of rows. To get a true overall mean, AVG the raw rows, or compute SUM(...) / COUNT(...) yourself.

Exercises

2.1. In a single query, compute COUNT(*), SUM, AVG, MIN, and MAX of orders.amount, and confirm each number by hand against the twelve rows.

2.2. For each of the five aggregates, write one business question it answers about the shop, phrased in plain language.

2.3. Explain, with a concrete example, a situation in which COUNT(*) and COUNT(col) return different numbers.

Answers. (1) The query is the profiling example above; the twelve amounts sum to 4020, and $4020 / 12 = 335$, matching total and mean. (2) e.g. COUNT → “how many orders did we take?”; SUM → “what was total revenue?”; AVG → “what is

a typical order worth?"; MIN → "what was our smallest order?"; MAX → "what was our biggest?". (3) When the counted column contains NULLs: on a table of 6 leads where one has no country, `COUNT(*) = 6` but `COUNT(country) = 5`.

3. Joins

3.1. Putting the split-up tables back together

Chapter 1 split data across tables to avoid repetition. A **join** is how you bring the pieces back together for a query: it combines rows from two tables by **matching values** in a related column — almost always a foreign key in one table meeting the primary key in the other. This is exactly the `df.merge` you used in pandas.

Example — Attaching each order to its customer. The orders table stores only a `customer_id`. To see the customer’s **name** beside each order — here, every order placed by a French customer — join the two tables on that key:

```
SELECT o.id, c.name, o.amount
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.id
WHERE c.country = 'France'
ORDER BY o.id;
```

id	name	amount
4	Bruno	200
5	Bruno	450
6	Bruno	380
9	Farah	600
10	Farah	520

AS o and AS c are **table aliases** — short nicknames so you can write `o.id` and `c.name`. The ON clause states the matching rule: pair an order with the customer whose `id` equals the order’s `customer_id`.

Remark. A bare JOIN, as written above, means INNER JOIN — the two are exactly synonyms, and you will see the four named join types in the next section. Also, **always qualify columns with the table alias** (`o.amount`, `c.name`) in a join. If both tables have a column called `id` and you write a bare `id`, the database cannot tell which you mean and raises an “ambiguous column” error. Qualifying every column also makes the query far easier to read.

3.2. The four join types

The tables rarely match perfectly: some customers have no orders, and (in a messy database) some orders might reference a customer that no longer exists. What should happen to those **unmatched** rows? That single decision is what distinguishes the four join types.

Definition 4 (INNER, LEFT, RIGHT, FULL OUTER).

- **INNER JOIN** keeps only rows that have a match in **both** tables. Unmatched rows on either side are dropped.
- **LEFT JOIN** keeps **all** rows of the left table; where the right table has no match, its columns come back NULL.
- **RIGHT JOIN** keeps **all** rows of the right table; unmatched left columns are NULL.
- **FULL OUTER JOIN** keeps **all** rows from **both** tables; unmatched columns on whichever side are NULL.

left circle = left-table rows, right circle = right-table rows, overlap = matched

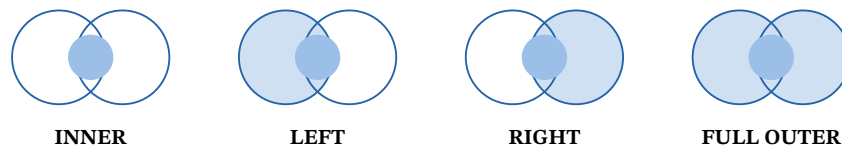


Figure 3: The four joins differ only in which unmatched rows they keep. INNER keeps the overlap (matched rows only); LEFT adds the unmatched left rows; RIGHT adds the unmatched right rows; FULL OUTER keeps everything. Shaded = returned by the join.

To see all four at once, build a miniature customers/orders pair — small enough to read whole. Because these two tables reuse the names customers and orders, create them in a **separate** throw-away database (say `joins.db`, made exactly as you made `shop.db`) so they do not collide with your shop:

```
CREATE TABLE customers (id INTEGER PRIMARY KEY, name TEXT);
CREATE TABLE orders    (id INTEGER PRIMARY KEY, customer_id INTEGER);
INSERT INTO customers VALUES (1, 'Amara'), (2, 'Bruno'), (3, 'Chen');
INSERT INTO orders    VALUES (10, 1), (11, 2), (12, 99);
```

Customer **Chen** has never ordered (an unmatched **left** row), and order 12 names customer_id 99, which exists in no customer row — an **orphan** (an unmatched **right** row):

id	name
1	Amara
2	Bruno
3	Chen

id	customer_id
10	1
11	2
12	99

The same SELECT, with only the join keyword changed, gives four different results. Here is the LEFT JOIN form; swap LEFT for INNER, RIGHT, or FULL OUTER to get the others:

```
SELECT c.name, o.id AS order_id
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id
ORDER BY c.name;
```

Watch Chen and order 12 appear and disappear across the four results:

INNER JOIN

name	order_id
Amara	10
Bruno	11

LEFT JOIN

name	order_id
Amara	10
Bruno	11
Chen	

RIGHT JOIN

name	order_id
	12
Amara	10
Bruno	11

FULL OUTER JOIN

name	order_id
	12
Amara	10
Bruno	11
Chen	

INNER returns only Amara and Bruno (the matched pair). LEFT adds Chen with a blank order. RIGHT adds the orphan order 12 with a blank name. FULL OUTER keeps both extras. A blank cell in these tables is a NULL — the join’s way of saying “no match on this side”.

3.3. Which join, and why it matters

The choice is not cosmetic — it changes both the rows you get and the **numbers you compute from them**.

Example — Listing every customer, even the inactive ones. “List all customers and their total spend, including those who have never ordered.”

```
SELECT c.name, COALESCE(SUM(o.amount), 0) AS total_spend
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id
GROUP BY c.id, c.name
ORDER BY total_spend DESC;
```

name	total_spend
Farah	1120
Bruno	1030
Gita	960
Amara	670
Diego	240
Chen	0

A `LEFT JOIN` from customers guarantees every customer appears — including **Chen**, who has no orders. A customer with no orders gets `NULL` for the order columns; `SUM` of nothing is `NULL`, so `COALESCE(..., 0)` turns it into a clean 0. Use an `INNER JOIN` here and Chen vanishes from the report entirely — a silent undercount.

Now the key analytical query. Chapter 1 could only group orders by the opaque `customer_id`; joining to customers lets you group by a **real attribute** — revenue per country:

```
SELECT c.country, SUM(o.amount) AS revenue
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.id
GROUP BY c.country
ORDER BY revenue DESC;
```

country	revenue
France	2150
Brazil	1630
Spain	240

Japan is absent: Chen is the only Japanese customer and has no orders, so an `INNER JOIN` drops the country entirely. If the report must list every country, that is again a job for a `LEFT JOIN` from customers.

Pitfall. The most dangerous join bug is **silent row loss**. An `INNER JOIN` quietly drops rows that don't match — so if a few orders have a `customer_id` that isn't in customers (like order 12 above), those orders disappear and your `SUM(amount)` is too low, with no error to warn you. When **every** row of one table must survive the join, reach for a `LEFT JOIN` from that table. A good habit: compare `COUNT(*)` before and after a join to see how many rows the match dropped.

3.4. Joining more than two tables

Real questions span several tables. You simply chain `JOIN` clauses; each adds one more table and one more `ON` condition. To show **what** each customer bought, bring in products too — every Brazilian customer's orders with the product name attached:

```
SELECT c.name, p.name AS product, o.amount
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.id
JOIN products AS p ON p.id = o.product_id
WHERE c.country = 'Brazil'
ORDER BY o.id;
```

name	product	amount
------	---------	--------

Amara	Bottle	120
Amara	Notebook	300
Amara	Backpack	250
Gita	Backpack	700
Gita	Notebook	260

Remark. SQLite gained `RIGHT JOIN` and `FULL OUTER JOIN` only in version 3.39 (2022); the queries above run on any current SQLite. On an older engine you would not miss them: `A RIGHT JOIN B` is just `B LEFT JOIN A` with the tables swapped, and a `FULL OUTER JOIN` is a `LEFT JOIN UNION`-ed with the unmatched rows of the other side. Master `LEFT JOIN` and you can always express the other three.

Remark. The Venn diagram in Figure 3 is a helpful picture, but not the whole truth: it suggests a join returns a **subset** of rows, but when one row on the left matches **several** on the right (a one-to-many link, the common case), the join **repeats** the left row once per match and can return **more** rows than either input. This is exactly why joining before you aggregate can inflate a `SUM` — another reason to check row counts around a join.

Exercises

3.1. Rewrite the per-country revenue query so that **every** country appears, Japan included with a revenue of 0. Which join, and which function, do you need?

3.2. On the miniature tables, predict the row count of each of the four joins **before** running them, then run them to check.

3.3. Using the orphan order 12, demonstrate the silent-row-loss pitfall: write one `INNER JOIN` and one `RIGHT JOIN` of customers and the miniature orders, and explain why their `COUNT(*)` differs.

Answers. (1) `SELECT c.country, COALESCE(SUM(o.amount), 0) AS revenue FROM customers c LEFT JOIN orders o ON o.customer_id = c.id GROUP BY c.country ORDER BY revenue`

`DESC`; — a `LEFT JOIN` from customers keeps Japan, and `COALESCE` turns its `NULL` sum into 0. (2) `INNER` → 2, `LEFT` → 3, `RIGHT` → 3, `FULL OUTER` → 4, as printed above. (3) The `INNER JOIN` drops order 12 (no matching customer), so it counts 2 rows; the `RIGHT JOIN` keeps every order including the orphan, so it counts 3. The missing row is the silent loss.

4. Subqueries and CTEs

4.1. When one query is not enough

Some questions need a query **whose answer feeds another query**. “Which orders are above average?” first needs the average, then the rows beating it. SQL gives two tools for this: the **subquery** (a query nested inside another) and the **CTE** (a named query you define up front). They can express the same things; CTEs are usually far more readable.

4.2. Subqueries

The quickest way to understand a subquery is to meet one where you cannot avoid it. “Which orders are above the overall average amount?” needs the average before it can keep the rows beating it:

Example — Orders above the overall average.

```
SELECT id, customer_id, amount
FROM orders
WHERE amount > (SELECT AVG(amount) FROM orders);
```

id	customer_id	amount
5	2	450
6	2	380
9	5	600
10	5	520
11	6	700

The inner `SELECT AVG(amount) FROM orders` runs first and returns one number (335); the outer query then keeps every order beating it. You could not do this with a plain `WHERE amount > AVG(amount)` — an aggregate is not allowed directly in `WHERE` — so the subquery is what lets you compare a row against a summary of the whole table.

That subquery sat in `WHERE`, but it is not the only place one can go.

Definition 5 (Subquery). A **subquery** is a `SELECT` written inside another query. It can appear:

- in `WHERE`, to compare against a computed value or a set of values;
- in `FROM`, as a temporary table the outer query reads from;
- in the `SELECT` list, to compute a value per row.

A subquery that returns a **list** pairs with `IN` — here, the customers who have placed at least one large order:

```
SELECT name
FROM customers
WHERE id IN (SELECT customer_id FROM orders WHERE amount > 500);
```

name
Farah
Gita

The two queries above put the subquery in WHERE, but the definition promised two more positions. A subquery in FROM stands in for a **table** — a **derived table** the outer query reads from — which is what you reach for when you must aggregate, then aggregate again. AVG(amount) averages the raw orders; to instead average each **customer's total**, total the customers first (the derived table), then average those totals:

```
SELECT ROUND(AVG(total), 2) AS avg_customer_total
FROM (
  SELECT customer_id, SUM(amount) AS total
  FROM orders
  GROUP BY customer_id
) AS per_customer;
```

avg_customer_total
804.0

The inner query produces one row per customer **who has placed an order** — five rows here, since Chen has never ordered, exactly the silent omission Chapter 3 warned of. The outer query treats that result as a table called per_customer and averages its total column. The answer (804) is the average of those per-customer totals, and it sits well above the per-order AVG(amount) of 335 — a reminder from Chapter 2 that averaging totals and averaging rows are different questions. (Were you to LEFT JOIN from customers so Chen counts as a 0, the average over all six customers would instead be 670.)

Finally, a subquery in the SELECT list computes **one value per row**. Here each of the largest orders is shown beside the whole table's average and its distance from it:

```
SELECT id, amount,
       amount - (SELECT AVG(amount) FROM orders) AS vs_avg
FROM orders
ORDER BY amount DESC
LIMIT 4;
```

id	amount	vs_avg
----	--------	--------

11	700	365.0
9	600	265.0
10	520	185.0
5	450	115.0

The `(SELECT AVG(amount) FROM orders)` runs once and its single value is attached to every row, so `vs_avg` reads as “how far this order sits above the typical order”.

4.3. Common Table Expressions (CTEs)

As questions grow, nesting subqueries inside subqueries becomes unreadable — you have to read from the inside out. A **CTE** fixes this by letting you **name** a query and then refer to it by name, top to bottom, like defining a variable before using it.

Definition 6 (Common Table Expression (WITH)). A **CTE** is a named, temporary result set defined with a `WITH` clause at the start of a query and referenced by name in what follows. It exists only for the duration of that one query. Several CTEs are separated by commas, and each may build on the ones before it — letting you decompose a complex query into readable, named steps.

Example — The same question, as a readable pipeline. “Which countries have above-average total revenue?” — solved in two named steps:

```
WITH country_totals AS (
  SELECT c.country, SUM(o.amount) AS revenue
  FROM orders AS o
  JOIN customers AS c ON o.customer_id = c.id
  GROUP BY c.country
)
SELECT country, revenue
FROM country_totals
WHERE revenue > (SELECT AVG(revenue) FROM country_totals)
ORDER BY revenue DESC;
```

country	revenue
France	2150
Brazil	1630

`country_totals` is computed once and named; the main query reads from it twice (once to list, once to average) as if it were a real table. France and Brazil clear the average of the three country totals; Spain does not.



Figure 4: A CTE names an intermediate result so the query reads as a left-to-right pipeline — raw rows → grouped totals → filtered final answer — instead of an inside-out nest of subqueries.

Remark. Reach for a subquery for a quick, one-off nested value (often in WHERE). Reach for a CTE when the query has several steps, when an intermediate result is used **more than once** (as `country_totals` is above), or simply when a name would make the logic clearer. Readable SQL is maintainable SQL, and CTEs are the main tool for keeping complex queries readable.

Exercises

4.1. Write “which orders are above the overall average amount?” first as a subquery, then rewrite it as a CTE that names the average. Explain when you would prefer each form.

4.2. Using a subquery with `IN`, list the **names** of customers who have never placed an order over 500. (Hint: `NOT IN`, or `IN` on the complement.)

4.3. Using a subquery in `FROM` (a derived table), compute the average number of orders **per customer**: count each customer’s orders in the inner query, then average those counts in the outer one. Explain why a plain `COUNT(*)` on orders cannot answer this.

Answers. (1) Subquery: `SELECT id, amount FROM orders WHERE amount > (SELECT AVG(amount) FROM orders);`. CTE: `WITH stats AS (SELECT AVG(amount) AS a FROM orders) SELECT id, amount FROM orders, stats WHERE amount > stats.a;`. Prefer the subquery for a single throwaway value; prefer the CTE when the value is reused or the query grows several steps. (2) `SELECT name FROM customers WHERE id NOT IN (SELECT customer_id FROM orders WHERE amount > 500);` returns Amara, Bruno, Chen, Diego. (3) `SELECT ROUND(AVG(n), 2) FROM (SELECT customer_id, COUNT(*) AS n FROM orders GROUP BY customer_id) AS per_customer;` returns 2.4 — averaged over the five customers who have actually ordered (Chen, with none, is again absent). A plain `COUNT(*)` counts rows (orders), giving 12; the per-customer average needs the counts formed first, in the derived table, before they can be averaged.

5. Creating and manipulating tables

5.1. Defining where the data lives

So far you have only **read** data. To build a database you must also **create** tables and **change** their contents. These four statements — CREATE, INSERT, UPDATE, DELETE — are the write side of SQL.

Because UPDATE and DELETE change data for good, it is wise to practise them on a **throwaway** table rather than on data you care about. So build one more table in your shop database — a small contacts table, kept separate from the customers, orders, and products you have been querying — and do all the writing in this chapter there:

Example — Defining a table.

```
CREATE TABLE contacts (
  id      INTEGER PRIMARY KEY,
  name    TEXT NOT NULL,
  email   TEXT UNIQUE,
  country TEXT
);
```

Each line declares a column, its **type**, and optional **constraints**: `id` is the primary key (unique, not null, identifies the row); `name` may not be NULL; `email` must be unique across all rows; `country` is free to be anything, including missing.

Definition 7 (Column types and constraints). A **type** declares what kind of value a column is meant to hold — INTEGER, REAL (decimal), TEXT, and in SQLite the catch-all BLOB. A **constraint** is a rule the database **enforces** on every write:

- PRIMARY KEY — unique row identifier;
- NOT NULL — a value is required;
- UNIQUE — no two rows may share this value;
- DEFAULT `v` — value to use when none is given;
- FOREIGN KEY (`col`) REFERENCES `other(id)` — `col` must match a row in `other`.

Constraints are the database protecting your data's integrity **for** you — a guarantee an in-memory DataFrame cannot give.

Pitfall. In most databases a column's **type** is strictly enforced. In SQLite — the engine this course uses — a declared type is only an **affinity**: SQLite will happily store the text 'hello' in a column you declared INTEGER. Rely on your NOT NULL/UNIQUE **constraints** for integrity, and do not assume the type alone will reject bad data. (Foreign-key checks, likewise, are off unless you run `PRAGMA foreign_keys = ON`.)

5.2. Inserting, updating, deleting

Start from the empty contacts table just created, add two rows, then change and remove them — watching the table after each statement.

INSERT adds rows:

```
INSERT INTO contacts (id, name, email, country) VALUES
  (1, 'Alice', 'alice@example.com', 'France'),
  (2, 'Bob', 'bob@example.com', 'Spain');
```

id	name	email	country
1	Alice	alice@example.com	France
2	Bob	bob@example.com	Spain

UPDATE changes existing rows in place:

```
UPDATE contacts
SET country = 'Germany'
WHERE id = 1;
```

id	name	email	country
1	Alice	alice@example.com	Germany
2	Bob	bob@example.com	Spain

DELETE removes rows:

```
DELETE FROM contacts
WHERE id = 2;
```

id	name	email	country
1	Alice	alice@example.com	Germany

Pitfall. An **UPDATE** or **DELETE** without a **WHERE** clause hits every row in the table. `DELETE FROM contacts;` empties the table; `UPDATE contacts SET country = 'France';` makes everyone French. This is the single most destructive beginner mistake. Before running an **UPDATE/DELETE**, write the **WHERE** first, and ideally run the same **WHERE** inside a **SELECT** to see exactly which rows you are about to change before you change them.

Remark. Statements that change data are wrapped in **transactions**. A transaction groups several changes so they either **all** succeed or **all** fail together, leaving the database consistent. When you work from Python (next chapter), changes are not permanent until you `commit` — until then you can `rollback` and undo them. This safety net is another thing a database gives you that editing a CSV by hand does not.

Exercises

5.1. CREATE a table with a primary key, a NOT NULL column, and a UNIQUE column. INSERT several rows, UPDATE one, and DELETE another, checking the table after each.

5.2. Attempt to INSERT two rows sharing the same value in the UNIQUE column. What does the database do, and why is that the point of the constraint?

5.3. Describe exactly what would happen if you ran your UPDATE or DELETE from the first exercise **without** its WHERE clause.

Answers. (1) See the INSERT/UPDATE/DELETE sequence above; run each and `SELECT *` between them. (2) The second INSERT is rejected with a uniqueness error and no row is added — the constraint stops duplicate values from ever entering the table. (3) A WHERE-less UPDATE rewrites the chosen column in **every** row; a WHERE-less DELETE empties the table.

6. Database types: relational, warehouse, flat file

6.1. Different jobs, different stores

A relational database is not the only place to keep data, nor always the right one. This chapter is about telling three kinds apart and saying when each fits. The deciding factor is the **workload**: many small reads and writes, or large read-mostly analytical scans, or simple portable storage. Three everyday scenarios show the split:

Example — Matching store to scenario.

- A web shop recording each order the instant a customer checks out → **relational database** (frequent single-row writes, integrity constraints).
- The analytics team computing five years of sales trends every morning → **data warehouse** (massive aggregation, read-mostly).
- A 200-row lookup table you send to a colleague → **flat file** (CSV; no engine needed).

Each scenario points at a different kind of store. Named precisely:

Definition 8 (Three kinds of data store).

- A **relational database** (OLTP — online **transaction** processing) stores data in related tables and is tuned for many fast reads and writes of **individual rows**: an app recording orders as they happen. Examples: PostgreSQL, MySQL, SQLite.
- A **data warehouse** (OLAP — online **analytical** processing) stores large volumes of historical data tuned for queries that **scan and aggregate across millions of rows**: “total revenue per region per quarter for five years”. Examples: BigQuery, Snowflake, Redshift.
- A **flat file store** keeps data in plain files (CSV, JSON, Parquet) with **no query engine** of its own. Simple, portable, and ideal for small datasets, exchange between tools, or one-off storage.

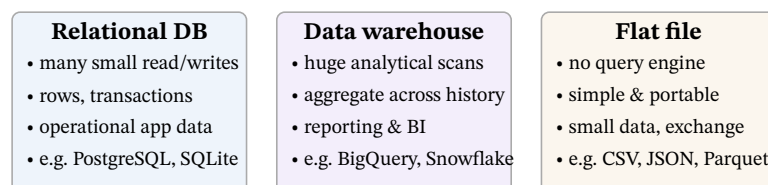


Figure 5: Choosing a store by workload: a relational database for live transactional row-level read/writes, a data warehouse for large-scale analytical aggregation over history, a flat file for small, portable, or one-off data.

Remark. The lines blur in practice — SQLite is a relational database that lives in a single file, and modern warehouses speak SQL just like relational databases do. The point is not rigid taxonomy but **matching the tool to the workload**: row-level transactions, large-scale analytics, or simple portable storage.

Exercises

6.1. For three scenarios of your own, state whether a relational database, a data warehouse, or a flat file store is the right choice, and justify each by the workload it implies.

6.2. The shop's live checkout runs on a relational database, yet the finance team copies its data nightly into a warehouse to run reports. Explain, in workload terms, why one store is not made to do both jobs.

Answers. (1) Any defensible mapping, e.g.: a banking app's transactions → relational (many small integrity-critical writes); a decade of sensor readings analysed monthly → warehouse (huge read-mostly scans); a config table shared over email → flat file (tiny, portable). (2) The checkout does many tiny row-level writes that a relational database is tuned for; the reports scan and aggregate millions of historical rows, which would bog down the live system — so the analytical workload is moved to a warehouse built for it.

7. Connecting to SQLite from Python

7.1. SQL where you already work

A query is only useful if you can run it from your analysis code and feed the result into pandas, a model, or a report. **SQLite** is a complete relational database that lives in a single file and needs no server, and Python's built-in `sqlite3` module talks to it directly — no installation required. This is the bridge between this course and every Python-based course around it.

Example — A full round-trip.

```
import sqlite3

conn = sqlite3.connect("shop.db")      # open the database file
cursor = conn.cursor()

cursor.execute(
    "SELECT name, country FROM customers WHERE country = ?",
    ("France",),                       # values fill the ? placeholders
)
for name, country in cursor.fetchall(): # pull the rows back into Python
    print(name, country)

conn.close()
```

`execute` sends the query; `fetchall` pulls the rows back into Python. Running it against `shop.db` prints the two French customers:

```
Bruno France
Farah France
```

The code turns on two objects — a connection and a cursor. Named precisely:

Definition 9 (Connection and cursor). A **connection** (`sqlite3.connect("file.db")`) is your session with the database. A **cursor** (`conn.cursor()`) is the object you use to **execute** statements and **fetch** their results: `execute(sql)` runs a statement; `fetchone()` returns the next result row, `fetchall()` returns all remaining rows as a list of tuples.

Pitfall. Never build SQL by string-formatting user input. Writing `f"... WHERE name = '{name}'"` is the classic **SQL injection** hole. Feed it a name of `x' OR '1'='1'`, and the query becomes `... WHERE name = 'x' OR '1'='1'` — a condition true for **every** row, so the filter is bypassed and the whole table leaks. Always pass values as parameters with `?` placeholders and a tuple — `execute("... WHERE country = ?", (country,))` — so the database treats them strictly as data, never as code. (Note the one-element tuple `(country,)` **needs** the comma.)

You may have seen the notorious `'; DROP TABLE customers; --` payload. It does **not** work through `sqlite3` here: `cursor.execute` runs a single statement and refuses a second, so the injected `DROP` is rejected outright. But injection needs no second statement — the one-line `OR '1'='1'` above already does the damage — so parameterise regardless of the driver.

Remark. In data work the most common shortcut is to skip the cursor entirely and let pandas do the round-trip: `pd.read_sql_query("SELECT ...", conn)` returns the result straight as a DataFrame, and `df.to_sql("table", conn)` writes one back. Under the hood it is the same `sqlite3` connection:

```
import pandas as pd, sqlite3

conn = sqlite3.connect("shop.db")
df = pd.read_sql_query(
    "SELECT c.country, SUM(o.amount) AS revenue "
    "FROM orders o JOIN customers c ON o.customer_id = c.id "
    "GROUP BY c.country ORDER BY revenue DESC",
    conn,
)
conn.close()
print(df)
```

```
country  revenue
0  France    2150
1  Brazil    1630
2   Spain     240
```

Exercises

- 7.1. From Python, connect to `shop.db` with `sqlite3`, run a **parameterised** query with a `?` placeholder that lists orders above a chosen amount, and print the rows.
- 7.2. Explain, with the injection example above, why the parameterised form is safer than building the SQL string with an f-string.
- 7.3. Load the per-country revenue query into a DataFrame with `pd.read_sql_query`, and confirm the numbers match the pure-SQL result from Chapter 3.

Answers. (1) `cur.execute("SELECT id, amount FROM orders WHERE amount > ?", (400,))` then `print(cur.fetchall())`. (2) A parameter is passed to the database as **data**: the string `x' OR '1'='1'` is matched literally against name — no customer is called that, so nothing extra returns. An f-string instead splices it into the query text, where the `OR '1'='1'` becomes part of the condition and matches every row. (3) The DataFrame shows France 2150, Brazil 1630, Spain 240 — identical to the pure-SQL per-country revenue result in Chapter 3.

8. Data quality at the query level

8.1. Clean the data where it lives

In pandas, you learned to clean a DataFrame after loading it. Often it is better to catch problems **in the query itself**, at the source. Three issues recur: duplicates, NULLs, and inconsistent strings. To see them, build one more table — a deliberately messy leads table — the same way you built shop.db:

```
CREATE TABLE leads (
  id      INTEGER PRIMARY KEY,
  name    TEXT,
  email   TEXT,
  country TEXT
);
INSERT INTO leads (id, name, email, country) VALUES
(1, 'Alice', 'alice@example.com', 'France'),
(2, 'Bob',   'bob@example.com',   'Spain '),
(3, 'Alice', 'alice@example.com', 'france'),
(4, 'Cara',  'cara@example.com',   NULL),
(5, 'Dan',   'dan@example.com',    ' FRANCE'),
(6, 'Bob',   'bob@example.com',    'Spain');
```

Its six rows:

id	name	email	country
1	Alice	alice@example.com	France
2	Bob	bob@example.com	Spain
3	Alice	alice@example.com	france
4	Cara	cara@example.com	
5	Dan	dan@example.com	FRANCE
6	Bob	bob@example.com	Spain

Two things are already wrong and invisible: rows 1, 3, and 5 are the same country typed three ways, and row 2's country is 'Spain ' with a **trailing space** you cannot see but the database can.

8.2. Duplicates

Duplicate rows inflate counts and totals. Find them by grouping on the columns that **should** be unique and counting:

```
SELECT email, COUNT(*) AS n
FROM leads
GROUP BY email
HAVING COUNT(*) > 1;
```

email	n
alice@example.com	2
bob@example.com	2

Both `alice@example.com` and `bob@example.com` appear twice. To return a deduplicated result instead, `SELECT DISTINCT` drops exact-duplicate rows:

```
SELECT DISTINCT name, email
FROM leads
ORDER BY name;
```

name	email
Alice	alice@example.com
Bob	bob@example.com
Cara	cara@example.com
Dan	dan@example.com

8.3. NULL handling

`COUNT` already reveals a NULL: counting rows versus counting the country column disagree, because one lead has no country recorded.

```
SELECT COUNT(*)      AS n_rows,
       COUNT(country) AS n_with_country
FROM leads;
```

n_rows	n_with_country
6	5

Definition 10 (What NULL means). NULL is not zero and not an empty string — it means **unknown** / **missing**. Any arithmetic or comparison with NULL yields NULL (effectively “unknown”), so `NULL = NULL` is **not** true. You therefore test for it with `IS NULL` and `IS NOT NULL`, `never = NULL`, and substitute a value with `COALESCE(col, replacement)`, which returns the first non-NULL argument.

```
SELECT name, COALESCE(country, 'Unknown') AS country
FROM leads
WHERE country IS NULL;
```

name	country
Cara	Unknown

Pitfall. The deepest NULL trap is that WHERE country <> 'France' **excludes** the rows where country is NULL — because NULL <> 'France' is NULL, not true. Cara, whose country is unknown, silently drops out. This query loses her:

```
SELECT name, country
FROM leads
WHERE country <> 'France';
```

name	country
Bob	Spain
Alice	france
Dan	FRANCE
Bob	Spain

If you want non-French leads **including** those with no country recorded, you must say so explicitly with OR country IS NULL, and Cara reappears:

```
SELECT name, country
FROM leads
WHERE country <> 'France' OR country IS NULL;
```

name	country
Bob	Spain
Alice	france
Cara	
Dan	FRANCE
Bob	Spain

8.4. Inconsistent string formats

The same value typed three ways — 'France', 'france', ' FRANCE' — counts as three distinct groups, scattering your aggregates. Normalise with string functions inside the GROUP BY:

```
SELECT LOWER(TRIM(country)) AS country, COUNT(*) AS n
FROM leads
GROUP BY LOWER(TRIM(country))
ORDER BY n DESC;
```

country	n
france	3
spain	2
	1

TRIM strips leading/trailing whitespace (rescuing 'Spain ' and ' FRANCE'), and LOWER/UPPER unify case, so the three France variants collapse into one group of three and the two Spain variants into one of two. The third row — blank, with n of 1 — is Cara: TRIM and LOWER applied to a NULL are still NULL (LOWER(TRIM(NULL)) = NULL), so, exactly as with the NULL grouping seen above, her unknown country forms its own group and prints blank rather than folding into any country.

Exercises

8.1. On the leads table, write one query that finds duplicate emails and one that returns a duplicate-free list of (name, email) pairs.

8.2. Show a <> filter that silently drops the NULL-country lead, then fix it so the row reappears.

8.3. Write a query that reports how many leads belong to each country **after** normalising case and whitespace, and explain why grouping on the raw column gives the wrong answer.

Answers. (1) Duplicates: SELECT email, COUNT(*) FROM leads GROUP BY email HAVING

COUNT(*) > 1;. Distinct pairs: SELECT DISTINCT name, email FROM leads;. (2) Bug: ... WHERE country <> 'France' (drops Cara); fix: ... WHERE country <>

'France' OR country IS NULL. (3) SELECT LOWER(TRIM(country)), COUNT(*) FROM

leads GROUP BY LOWER(TRIM(country)); gives france 3, spain 2, and a blank group of 1 for Cara's NULL country (which TRIM/LOWER leave NULL). Grouping on the raw column instead splits France into three one-row groups because 'France', 'france', and ' FRANCE' are three different strings.

9. From business question to SQL

9.1. The translation skill

The whole course converges here: turning a question asked in plain language into a query, running it, and reporting the answer in business terms. This is a repeatable procedure, not a flash of insight.

Definition 11 (Question → query in five steps).

1. **Identify the tables** that hold the relevant facts (and how they join).
2. **Filter the rows** the question restricts to → WHERE.
3. **Decide the grouping** — is the answer one number, or one per category? → GROUP BY.
4. **Choose the summary** — count, total, average, extreme? → an aggregate.
5. **Order and limit** for “top/bottom N”, then **interpret** the result in the words of the original question.

Example — “Which three countries generated the most revenue in 2025?”. Walk the steps: the facts are in orders, joined to customers for the country; restrict to 2025 (WHERE); the answer is one number **per country** (GROUP BY country); the summary is total revenue (SUM); we want the top three (ORDER BY ... DESC LIMIT 3).

```
SELECT c.country, SUM(o.amount) AS revenue
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.id
WHERE o.order_date >= '2025-01-01'
GROUP BY c.country
ORDER BY revenue DESC
LIMIT 3;
```

country	revenue
France	1950
Brazil	1510
Spain	90

Interpreting the result: the three rows are the firm’s largest markets by revenue in 2025 — where, in plain terms, the money came from that year. France led with 1950, Brazil followed with 1510, and Spain trailed far behind at 90. A query is only finished once you can say what its rows **mean** to the business, not merely that they appeared.

Pitfall. A query that runs is not the same as a query that is **correct**. The dangerous failures are silent: an INNER JOIN that dropped unmatched rows, a NULL that skewed an AVG, a missing WHERE that counted every year instead of one. Sanity-check every answer — does the row count

look plausible? Does the total roughly match a quick estimate you can make in your head? An answer you cannot defend in business terms is not an answer yet.

Exercises

9.1. Translate “who is our single highest-spending customer, and how much have they spent?” into SQL using the five-step procedure, execute it, and write one sentence interpreting the result.

9.2. Translate a business question of your own into SQL, execute it, and write two sentences interpreting the result in business terms — including one sanity check you ran on the answer.

Answers. (1) `SELECT c.name, SUM(o.amount) AS spend FROM orders o JOIN customers c ON o.customer_id = c.id GROUP BY c.id ORDER BY spend DESC LIMIT 1;` returns Farah, 1120 — our most valuable customer by total spend. (2) Answers vary; a good one names the tables and join, states the filter/grouping/aggregate, and reports a check such as “the per-country totals sum to 4020, matching `SUM(amount)` over all orders.”

10. Indexing and query performance

10.1. When the table gets big

On a thousand rows every query is instant. On ten million, a query that filters on an unindexed column must read **every** row to find the matches — a **full table scan**. An **index** is the database's answer, and the trade-offs it carries are the last idea the course needs.

Definition 12 (Index). An **index** is an auxiliary data structure (typically a balanced tree) that keeps the values of one or more columns in sorted order, with pointers to the rows that contain them. It lets the database **jump straight** to the matching rows instead of scanning the whole table — turning a search from $O(n)$ (read every row) into roughly $O(\log n)$ (descend a tree).

Remark (reading the $O(\cdot)$ notation). $O(n)$ and $O(\log n)$ are a compact shorthand for **how the work grows as the table grows** — with n the number of rows. $O(n)$ means the effort grows in step with n : double the rows and you double the work, because you read every one. $O(\log n)$ means it grows like the **logarithm** of n — far slower: each tenfold increase in rows adds only a handful of extra steps. That gap is the whole case for an index. At ten million rows, an $O(n)$ scan touches ten million rows while an $O(\log n)$ descent touches about two dozen.

You can **see** the difference the engine makes. `EXPLAIN QUERY PLAN` reports how a query will run. Before any index, filtering orders on `customer_id` is a full scan:

```
EXPLAIN QUERY PLAN
SELECT * FROM orders WHERE customer_id = 2;
```

```
QUERY PLAN
  |--SCAN orders
```

Add an index on that column, and ask again:

```
CREATE INDEX idx_orders_customer ON orders(customer_id);
```

```
EXPLAIN QUERY PLAN
SELECT * FROM orders WHERE customer_id = 2;
```

```
QUERY PLAN
  |--SEARCH orders USING INDEX idx_orders_customer (customer_id=?)
```

`SCAN orders` has become `SEARCH orders USING INDEX ...` — the engine now jumps to the matching rows through the index instead of reading the whole table. On twelve rows the wall-clock difference is nothing; on ten million it is the difference between instant and unusable.

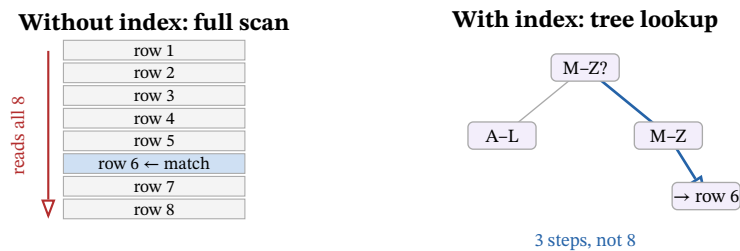


Figure 6: Searching for one value. Without an index the database reads every row (a full scan, $O(n)$). With an index it descends a sorted tree, touching only a few nodes ($O(\log n)$) — a gap that widens enormously as the table grows.

10.2. The trade-off: indexes are not free

An index would seem to make everything faster, so why not index every column? Because an index is a **second copy** of the data that the database must **keep in sync**.

Definition 13 (What an index costs). An index speeds up **reads** that filter, join, or sort on the indexed column — but it **slows down writes** (INSERT, UPDATE, DELETE), because every write must also update the index, and it **consumes extra storage**. An index is a bet that the column will be searched far more often than it is written.

Example — When to create one, when not to. Worth indexing: the `customer_id` column of a million-row orders table that you constantly filter and join on — the read speed-up is huge and well worth the write cost.

Not worth it: an `is_active` boolean with only two values (an index barely narrows the search), a tiny 50-row lookup table (a full scan is already instant), or a column you never filter on (you pay the write and storage cost for no read benefit).

Pitfall. Do not reflexively index everything. On a write-heavy table, redundant indexes can make the system **slower overall** — every insert now maintains a stack of indexes nobody queries. Index the columns that appear in your WHERE and JOIN conditions on **large** tables, measure the effect with `EXPLAIN QUERY PLAN` (as above) to confirm the index is actually used, and add no more than the queries justify.

Exercises

10.1. Run `EXPLAIN QUERY PLAN` on a query that filters orders by `customer_id`, create an index on that column, and run it again. Describe how the plan changes.

10.2. Name one column in the shop database worth indexing and one not worth it, and justify each by how often it is filtered versus written.

Answers. (1) The plan changes from `SCAN orders` to `SEARCH orders USING INDEX ...`, as shown above — the engine switches from reading every row to jumping through the index. (2) Worth it: `orders.customer_id`, filtered and joined constantly. Not worth it: a two-value flag column, or `customers.signup_date` if you never filter on it — both pay write/storage cost for little or no read benefit.